

OpenRD 开源协作治理与 SOP 方案

1. 文档目的

本文档用于整理 OpenRD 长线公益技术协作项目在运营、组织治理、任务拆分、成员培养和项目交接方面的核心问题与初步解决方案。

OpenRD 是一个以患者真实需求为导向，通过专业技术开发对应工具、解决具体问题的公益项目管理平台。平台本身不仅服务外部需求，也可以成为自身建设的第一个长期运营案例：用 OpenRD 管理 OpenRD 的开发。

如果这套流程在平台自身建设中跑通，后续可以抽象为一套标准模板，作为平台内所有需求从提交、沟通、转化、拆分、开发、验收、运维到复盘的开源项目开发 SOP。

2. 当前核心问题

2.1 单点依赖问题

当前项目推进在较大程度上依赖项目负责人个人。负责人既要对接社区运营和发起项目的团队，又要管理参与开发的成员，还要承担产品设计、文档书写、资源协调、任务推进和交接准备。

这种模式短期效率高，但长期风险明显：

- 项目负责人实习、离开或精力下降后，项目可能失速。
- 需求、任务、技术方案和外部资源都集中在单人认知中，难以交接。
- 项目关键权限、沟通渠道和决策依据如果没有沉淀，会形成隐性风险。
- 外部协作者很难判断项目当前状态，也不容易接手具体工作。

2.2 成员流动问题

开源和公益项目中的成员常常会因为学业、实习、工作、兴趣变化或时间不足而阶段性离开，甚至直接退出。

这不是个别异常，而是长期公益协作项目必须默认存在的现实条件。

如果项目依赖“某个人一直有空、一直愿意做、一直能负责到底”，项目稳定性会很差。更合理的设计是：允许成员低频参与、阶段性退出，但每一次贡献都能被接住、被延续。

2.3 能力与时间差异问题

不同参与者的技术能力、产品理解、文档能力和可投入时间差异很大。

如果任务颗粒度过大，只适合少数能力强且时间充足的人参与；如果任务说明不清，新人无法独立开始；如果没有验收标准，贡献者做完后也很难判断是否真正完成。

因此任务必须被拆分为不同难度、不同耗时、不同技能要求的可认领单元。

2.4 启动团队与开放协作的边界问题

为了推进项目，可以邀请身边同学组成较完整的技术团队，完成项目早期基础建设。但需要注意边界：

- 如果启动团队变成封闭团队，外部贡献者只能围观，会影响开放协作的积极性。
- 如果完全不建立稳定启动团队，只依靠松散志愿参与，项目早期又可能推进缓慢。

更合理的定位是：启动团队负责打地基、建规则、做样板、保底质量，但项目治理、任务池和贡献入口保持开放。

3. 总体思路

OpenRD 不应长期依赖某一个产品经理或开发者持续推进，而应建立一套轻量组织治理结构。

核心思路是：

用组织机制降低单点依赖

用文档和 SOP 降低交接成本

用任务拆分降低参与门槛

用 Review 和验收保证质量

用教学体系吸收新人

用平台自身建设验证平台机制

项目自身可以作为 OpenRD 的内部示范项目。平台需要什么功能，就把这个需求也发布到平台中；产品经理负责转化，任务拆分中枢负责拆分，贡献者领取任务，维护者进行 Review，最终合并和发布。

这形成一个自举机制：

平台发现自身问题

→ 作为需求发布

→ 产品经理评估并转化任务

→ 拆成子任务

→ 共建者领取开发

→ 维护者审查合并

→ 教学组沉淀经验

→ 平台功能继续进化

4. 轻量组织治理结构

建议采用“项目管理委员会 + 能力中枢 + 维护者体系 + 贡献者任务池”的轻量结构。

需要强调的是，这套结构不应在项目一开始就完整铺开。若每个能力中枢都配置 2 人，仅四个中枢就需要 8 人管理团队，这对早期公益项目过重。更合理的方式是根据项目规模、成熟度和运营时间使用不同体积的组织模板，让组织结构随项目一起进化。

4.1 项目管理委员会

项目管理委员会负责项目方向和关键治理，不负责包办所有开发。

主要职责：

- 确定项目阶段目标和优先级。
- 对接社区运营、外部导师、公益组织和资源方。
- 管理关键资源和关键权限。
- 决定重大需求是否进入开发计划。
- 定期复盘项目进度和风险。
- 推动负责人交接和备份人培养。

委员会不应成为新的瓶颈。它的重点不是审批一切，而是建立规则、协调资源、处理争议、保证项目持续运转。

4.2 四个能力中枢

相比设置很多固定职位，更适合建立四个能力中枢。每个中枢可以由 1-3 名成员负责，并设置备份人。

4.2.1 资源中枢

负责解决“项目需要什么、谁能提供、资源在哪里”。

职责包括：

- 对接社区运营、患者需求方、外部专家和支持者。
- 维护项目资源清单，例如账号、服务器、域名、仓库、设计稿、文档、会议记录。
- 维护人员能力画像，例如前端、后端、产品、设计、测试、文档、运维。
- 协调外部支持，例如导师指导、技术赞助、学校资源和公益组织资源。
- 管理资源申请、使用和交接。

4.2.2 架构中枢

负责解决“项目怎么搭、质量如何保证、技术边界是什么”。

职责包括：

- 技术选型。
- 前后端架构设计。
- 数据模型设计。
- 接口规范。
- 代码规范。
- Review 标准。
- 发布流程。
- 安全和权限边界。
- 核心模块负责人和备份人安排。

架构中枢不一定写完所有代码，但必须保证项目不会因为贡献者水平差异而失控。

4.2.3 任务拆分中枢

负责解决“如何把一个大需求拆成可交付的小任务”。

职责包括：

- 将需求转化为任务。
- 将大任务拆成子任务。
- 标注难度、预计耗时、技能要求、优先级。
- 明确每个任务的背景、目标、输入、输出和验收标准。
- 维护任务池。
- 处理任务无人认领、超时、阻塞和交接。
- 识别新人友好任务。

这是 OpenRD 的核心能力之一。平台要解决的不是“发布一个模糊目标”，而是把真实需求拆成不同人可以参与的协作单元。

4.2.4 教学与成长中枢

负责解决“新人怎么加入、怎么从不会到能贡献”。

职责包括：

- 编写新人入门指南。
- 讲解项目结构。
- 制作 Git、PR、代码规范、接口联调等教程。
- 提供任务领取流程说明。
- 设计新人任务模板。
- 整理常见问题。
- 组织分享会、答疑和 Review 反馈。
- 建立成长路径：新人任务 -> 普通任务 -> 模块任务 -> 维护者。

公益项目不能只依靠高手。教学与成长中枢的存在，是为了让普通同学、新人、低频参与者也能产生有效贡献。

4.3 维护者体系

维护者负责代码质量、文档质量和最终合并。

维护者不等于唯一开发者，而是质量守门人。

主要职责：

- 审查 PR。
- 判断是否满足验收标准。
- 指导贡献者修改。
- 维护核心模块。
- 处理技术债和质量风险。
- 参与版本发布。

核心模块应至少有 1 名负责人和 1 名了解者，避免模块知识只掌握在一个人手里。

4.4 贡献者任务池

贡献者任务池面向所有愿意参与的人。

任务池应支持不同类型任务：

- 新人任务：1-3 小时内可完成，说明清晰，风险低。
- 普通任务：半天到 2 天可完成，适合有一定能力的贡献者。
- 核心任务：涉及架构、权限、状态流转等关键模块，必须由维护者或稳定成员负责。
- 探索任务：技术调研、方案评估、原型优化等。
- 文档任务：说明书、教程、交接文档、FAQ、PRD 校对。
- 测试任务：功能测试、验收用例、问题复现、兼容性检查。

任务池的价值在于让不同能力、不同时间的人都能找到合适的参与入口。

4.5 分阶段组织进化模型

不同规模的项目不应使用同一种组织架构。OpenRD 可以设计一套可进化的治理模板，让项目从 MVP 团队逐步成长为成熟的项目管理组织。

4.5.1 阶段一：MVP 承接团队

适用场景：

- 项目刚启动。
- 需求还需要验证。
- 贡献者数量较少。
- 目标是快速做出最小可用版本。

组织形态：

- 1 名项目负责人。
- 1 名产品/需求负责人。
- 1-3 名核心开发者。

核心目标：

- 快速理解需求。
- 搭建基础架构。
- 完成 MVP。
- 跑通从需求到任务、开发、验收的最小闭环。

这个阶段不需要完整委员会，也不需要四个中枢都配齐人员。重点是把项目做起来，并记录开发过程中的真实问题。

4.5.2 阶段二：MVP 团队转化为管理机构

适用场景：

- MVP 已经跑通。
- 项目进入持续迭代。
- 新贡献者开始加入。
- 任务需要持续拆分和分配。

组织形态：

原始 MVP 团队根据实际贡献和职责逐步转化为项目管理机构：

- 熟悉需求的人转为产品/需求负责人。
- 熟悉架构的人转为技术负责人。
- 熟悉任务拆解的人转为任务拆分负责人。
- 擅长文档和指导的人转为教学/文档负责人。
- 稳定参与 Review 的人成为维护者。

这个阶段的管理层不是凭空任命，而是从实际开发者中自然进化出来。因为他们参与过 MVP，理解项目上下文，也更适合承担后续规划、拆分、Review、验收和交接工作。

4.5.3 阶段三：中型项目治理结构

适用场景：

- 项目已有稳定用户或真实使用场景。
- 贡献者数量增加。
- 同时存在多个任务或子任务。
- 需要持续迭代、维护和答疑。

组织形态：

- 项目负责人。
- 产品负责人。
- 技术负责人。
- 任务拆分负责人。
- Review/维护者。
- 文档/教学负责人。

这个阶段可以开始形成轻量项目管理委员会，但委员会重点是规划、协调、复盘和处理风险，而不是审批所有细节。

4.5.4 阶段四：长期项目治理结构

适用场景：

- 项目生命周期较长。
- 贡献者频繁流动。
- 存在多个版本、多个模块或多个维护方向。
- 负责人需要阶段性退出或交接。

组织形态：

- 项目管理委员会。
- 资源中枢。
- 架构中枢。
- 任务拆分中枢。
- 教学与成长中枢。
- 维护者体系。
- 贡献者任务池。

此时负责人应逐步放权给项目管理委员会。项目负责人不再亲自推动所有事情，而是负责方向监督、关键资源协调和机制建设。委员会承担规划、任务分解、验收、交接和持续迭代。

4.5.5 组织进化原则

组织进化应遵循以下原则：

1. 先做 MVP，再做治理。
2. 先让项目跑起来，再逐步拆分管理职责。
3. 管理者优先从实际贡献者中产生。

4. 项目规模扩大后再增加组织层级。
5. 负责人必须逐步放权，而不是永久成为唯一决策点。
6. 每一次组织升级都应解决真实问题，而不是为了形式完整。

5. 反单点依赖机制

为了避免项目依赖单人，应建立以下规则：

1. 关键角色必须有备份人。
2. 核心模块必须至少两个人了解。
3. 关键决策必须记录到文档。
4. 关键权限不得只掌握在一个人手中。
5. 每个任务必须有负责人、状态、下一步和验收标准。
6. 长时间无进展的任务应释放或重新分配。
7. 成员退出前应完成交接记录。
8. 每月进行一次项目复盘和交接检查。

项目治理的目标不是要求每个人永远在线，而是让任何人的阶段性离开都不会让项目失去方向。

6. 平台二期：子任务系统设计

正式 PRD 当前已经覆盖需求提交、需求审核、任务转化、任务认领和队伍协作。二期建议重点增加子任务系统，用来支持更细颗粒度的开源协作。

6.1 为什么需要子任务系统

子任务系统解决以下问题：

- 大任务过大，新人不知道从哪里开始。
- 贡献者能力和时间不同，需要不同颗粒度任务。
- 产品经理和技术负责人需要更精确地跟踪进度。
- 成员退出时，需要知道哪些部分已完成、哪些仍阻塞。
- 任务验收需要明确到具体交付物。
- 平台自身建设也需要拆分为可认领、可 Review 的工作项。

6.2 子任务适用场景

以下情况建议必须拆分子任务：

- 主任务预计超过 2 天。
- 需要 2 个以上角色协作。
- 同时涉及前端、后端、设计、测试、文档或运维。
- 存在新人可参与的局部工作。
- 存在明确的阶段性交付物。
- 任务长期阻塞，需要重新拆解。

6.3 子任务核心字段

子任务建议包含以下字段：

字段	说明
所属主任务	关联主任务 ID
子任务标题	简洁描述要完成的工作
背景说明	为什么需要做这个子任务
目标	做到什么程度
交付物	需要交付的代码、文档、设计稿或测试结果
验收标准	判断是否完成的条件
任务类型	前端、后端、测试、文档、设计、运维、调研
难度	新人、普通、进阶、核心
预计耗时	例如 1 小时、半天、2 天
所需技能	Vue、接口、数据库、文档、测试等
优先级	P0、P1、P2
负责人	当前认领者
协作者	参与协作的人
Review 人	负责审查的人
验收人	产品或任务负责人
状态	待认领、进行中、待 Review、待验收、已完成、已阻塞、已释放
截止时间	计划完成时间
阻塞原因	无法推进时填写
关联文档	PRD、设计稿、接口文档、教程
关联 PR	代码提交或合并请求
交接记录	成员退出或转交时填写

6.4 子任务状态流转

建议状态流转如下：

待拆分

→ 待认领

→ 进行中

→ 待 Review

→ 待验收

→ 已完成

异常状态：

已阻塞
已释放
已取消
需重拆

说明：

- 待 Review：技术或文档质量等待审查。
- 待验收：技术已通过，但产品目标或业务验收未完成。
- 已释放：认领者长期无响应或主动退出，任务回到任务池。
- 需重拆：任务仍然过大或说明不清，需要重新拆分。

6.5 子任务难度分级

等级	描述	适合人群
新人	说明清晰、风险低、1-3 小时内可完成	刚加入的贡献者
普通	需要理解局部业务和代码，半天到 2 天	有基础能力的贡献者
进阶	涉及跨模块协作或较复杂逻辑	稳定贡献者
核心	涉及架构、权限、数据模型、状态流转	维护者或核心成员

6.6 子任务验收原则

每个子任务必须能被单人理解、认领、交付和验收。

合格子任务应满足：

- 背景清楚。
- 范围明确。
- 交付物明确。
- 验收标准明确。
- 预计耗时合理。
- 依赖关系可见。
- 有 Review 人或验收人。

如果一个子任务无法在 2 天内完成，应继续拆分。

7. 标准 SOP 模板

每套 SOP 建议统一使用以下格式：

适用场景：
负责人：
输入：
标准步骤：
输出：
验收标准：
异常处理：
相关模板：

统一格式有助于降低理解成本，也方便后续沉淀到平台功能中。

8. 核心 SOP 草案

8.1 需求接收 SOP

适用场景：

需求者提交真实需求后，产品经理进行初步处理。

负责人：

产品经理。

输入：

需求标题、需求描述、附件、联系方式、紧急程度、需求者背景。

标准步骤：

1. 判断需求是否属于平台服务范围。
2. 判断描述是否清晰。
3. 判断是否重复已有需求或任务。
4. 判断是否需要补充沟通。
5. 判断是否具备转化为任务的价值。
6. 标注当前状态：待沟通、可转化、重复、挂起、驳回。

输出：

需求处理结论和下一步动作。

验收标准：

每条需求都有明确状态、处理意见和负责人。

异常处理：

需求描述过于模糊时，进入需求沟通 SOP；长时间无回复时进入挂起或驳回流程。

8.2 需求沟通 SOP

适用场景：

需求信息不足，产品经理需要与需求者进一步澄清。

负责人：

产品经理。

输入：

原始需求、疑问点、需求者联系方式。

标准步骤：

1. 确认真实使用场景。
2. 确认目标用户是谁。
3. 确认当前痛点是什么。
4. 确认期望工具解决什么问题。
5. 确认是否有现有替代方案。
6. 确认最小可用版本范围。
7. 整理沟通记录。

输出：

补充后的需求说明、用户场景、MVP 范围和沟通记录。

验收标准：

需求可以被转化为明确任务，或者有充分理由驳回、挂起、合并。

异常处理：

需求者超过约定时间未回复，标记为挂起；多次沟通仍无法明确，标记为暂不立项。

8.3 任务转化 SOP

适用场景：

产品经理将有效需求转化为可开发任务。

负责人：

产品经理，必要时邀请技术负责人。

输入：

需求说明、沟通记录、附件、MVP 范围。

标准步骤：

1. 编写任务标题。
2. 补充任务背景。
3. 定义任务目标。
4. 定义交付物。
5. 定义验收标准。
6. 评估需要的角色和技能。
7. 设置优先级和计划完成时间。
8. 判断是否需要拆分子任务。
9. 发布到任务大厅。

输出：

可认领的主任务。

验收标准：

共建者能够理解任务背景、目标和完成标准。

异常处理：

如果任务过大，进入子任务拆分 SOP；如果技术可行性不明，进入技术调研任务。

8.4 子任务拆分 SOP

适用场景：

主任务预计超过 2 天，或需要多人、多角色协作。

负责人：

任务拆分中枢，产品经理与技术负责人共同参与。

输入：

主任务、需求说明、技术方案、验收标准。

标准步骤：

1. 明确主任务目标。
2. 拆出最小可交付版本。
3. 按前端、后端、测试、文档、设计、运维拆分工作。
4. 为每个子任务标注难度、预计耗时和所需技能。
5. 为每个子任务写清交付物和验收标准。
6. 标记新人友好任务。
7. 标记核心任务和必须由维护者负责的任务。
8. 提交技术负责人确认。

输出：

一组可认领、可 Review、可验收的子任务。

验收标准：

每个子任务都可以被单人理解、认领、交付和验收。

异常处理：

如果子任务仍然超过 2 天，应继续拆分；如果依赖关系不清，应先补充技术方案。

8.5 共建者认领 SOP

适用场景：

共建者希望参与任务或子任务。

负责人：

任务负责人或队长。

输入：

任务信息、贡献者资料、能力标签、可投入时间。

标准步骤：

1. 共建者阅读任务说明和验收标准。
2. 共建者申请认领或加入队伍。
3. 任务负责人确认其能力和时间是否匹配。
4. 确认后分配任务。
5. 约定首次反馈时间。
6. 记录负责人和协作者。

输出：

任务认领记录。

验收标准：

任务有人负责，且负责人知道下一步做什么。

异常处理：

超过约定时间无反馈，提醒一次；继续无反馈则释放任务。

8.6 代码提交与 Review SOP

适用场景：

贡献者完成开发工作并提交代码。

负责人：

技术维护者。

输入：

代码提交、PR、关联任务、测试说明。

标准步骤：

1. 贡献者提交 PR。
2. PR 关联任务或子任务。
3. 贡献者填写完成内容、测试方式和影响范围。
4. 维护者进行代码 Review。
5. 必要时要求修改。
6. 通过后进入产品验收。
7. 验收通过后合并。

输出：

合并后的代码或修改意见。

验收标准：

代码符合规范，功能符合任务目标，无明显安全和质量问题。

异常处理：

如果 PR 长时间无人处理，应转交其他维护者；如果贡献者无法继续修改，应由任务负责人决定是否接手或释放。

8.7 任务验收 SOP

适用场景：

任务或子任务完成后，需要确认是否达到要求。

负责人：

产品经理和技术维护者。

输入：

交付物、验收标准、测试结果、需求者反馈。

标准步骤：

1. 技术维护者确认代码或交付物质量。
2. 产品经理按验收标准检查业务目标。
3. 必要时邀请需求者试用或确认。
4. 记录问题和修改项。
5. 修改完成后再次验收。
6. 标记任务完成。

输出：

验收结论、问题记录、完成状态。

验收标准：

交付物满足预设目标，关键问题已解决，相关记录完整。

异常处理：

不满足验收标准时退回修改；需求发生明显变化时新建变更任务，不直接扩大原任务范围。

8.8 交接与退出 SOP

适用场景：

负责人、维护者、贡献者因实习、学业、工作或其他原因离开项目。

负责人：

项目管理委员会或对应中枢负责人。

输入：

当前负责事项、任务状态、权限清单、文档清单、未完成问题。

标准步骤：

1. 列出当前负责的任务和模块。
2. 标注每项工作当前状态。
3. 写明下一步建议。
4. 移交相关文档、账号、权限和资源。
5. 指定接手人。
6. 召开或记录一次交接确认。

7. 更新任务负责人和备份人。

输出：

交接记录和新的负责人信息。

验收标准：

接手人能根据文档继续推进，不需要依赖原负责人长期在线。

异常处理：

如果成员无法联系，项目管理委员会根据任务记录重新分配任务；权限由资源中枢统一清点和回收。

9. SOP 与平台功能的对应关系

这些 SOP 不只是管理文档，也可以直接反推平台二期功能设计。

SOP	对应平台能力
需求接收 SOP	需求管理状态、处理意见、负责人
需求沟通 SOP	需求详情沟通区、补充资料、挂起规则
任务转化 SOP	需求转任务表单、任务发布流程
子任务拆分 SOP	子任务系统、难度标注、技能要求
共建者认领 SOP	认领申请、队伍成员、首次反馈提醒
Review SOP	PR 关联、Review 人、待 Review 状态
验收 SOP	验收清单、验收记录、需求者反馈
交接 SOP	备份人、交接记录、任务释放

这意味着 OpenRD 的产品能力可以从真实项目运营 SOP 中自然生长出来。

10. 可复用的开源项目开发 SOP 模板

当这套流程跑通后，可以抽象为平台通用模板，适用于后续所有需求转化任务与项目开发运维。

10.1 通用流程

需求提交
→ 需求接收
→ 需求沟通
→ 需求评估
→ MVP 承接团队接手
→ 任务转化
→ 子任务拆分
→ 任务认领
→ 开发协作
→ Review
→ 验收
→ MVP 发布
→ MVP 团队转化为项目管理机构
→ 发布

10.2 每个项目必须具备的角色

- 项目负责人：负责整体目标、外部协调和风险管理。
- 产品负责人：负责需求理解、任务转化和验收标准。
- 技术负责人：负责架构、技术方案、Review 和发布。
- 任务负责人：负责某个主任务或子任务推进。
- 贡献者：负责具体实现。
- 验收人：负责确认交付物是否满足目标。
- 备份人：负责降低单点依赖。

10.3 每个项目必须具备的文档

- 项目说明。
- 需求说明。
- 任务拆分表。
- 子任务列表。
- 技术方案。
- 接口文档。
- 验收标准。
- 贡献指南。
- 交接记录。
- 复盘记录。

10.4 每个任务必须具备的信息

- 背景。
- 目标。
- 范围。
- 交付物。
- 验收标准。
- 负责人。
- Review 人。
- 状态。
- 截止时间。
- 阻塞原因。
- 关联文档和 PR。

10.5 每个阶段必须回答的问题

需求阶段：

- 这是谁的真实问题？
- 这个问题是否值得平台介入？
- 需求是否清晰？
- 是否需要补充沟通？

任务阶段：

- 最小可交付版本是什么？
- 需要哪些角色？
- 能否拆成子任务？
- 哪些任务适合新人？

开发阶段：

- 谁负责？
- 什么时候反馈？
- 怎么提交？
- 谁 Review？

验收阶段：

- 是否满足业务目标？
- 是否满足技术质量？
- 是否需要需求者试用？
- 是否需要进入运维？

复盘阶段：

- 哪些流程有效？
- 哪些任务拆得不好？
- 哪些文档缺失？
- 哪些机制需要进入平台功能？

11. 建议后续行动

11.1 先用 OpenRD 自身建设试运行

建议将 OpenRD 自身开发作为第一个内部试点项目：

- 建立项目管理委员会。
- 建立四个能力中枢。
- 选择一个真实平台改进需求。
- 按 SOP 转化任务。
- 拆分子任务。
- 开放给贡献者认领。

- 通过 Review 和验收完成交付。
- 记录整个过程并复盘。

11.2 先做轻量，不做重组织

早期不需要复杂组织架构。建议先明确最小组织：

- 项目负责人。
- 产品负责人。
- 技术负责人。
- 文档/教学负责人。
- 资源协调负责人。

每个角色尽量设置备份人。

更具体地说，早期可以先由一个小型 MVP 团队承接任务项目。这个团队负责快速验证需求、开发最小可用版本、沉淀最初的架构和文档。等 MVP 跑通后，再让原始开发团队根据各自实际职责转化为项目管理机构，继续承担规划、任务分解、Review、验收、教学和交接工作。

这种打法可以避免一开始就建立过重的管理层，也能让管理权来自真实贡献。项目管理委员会不是凭空出现的，而是由已经理解项目、开发过项目、承担过责任的人逐步进化出来。

11.3 项目负责人要逐步放权

项目负责人早期可以承担更多推进责任，但不能长期成为唯一决策点。随着项目进入持续迭代，负责人应逐步把以下权力交给项目管理委员会：

- 阶段规划权。
- 任务优先级决策权。
- 任务分配权。
- Review 和合并规则制定权。
- 验收组织权。
- 资源协调和权限管理权。
- 交接和成员退出处理权。

负责人最终应从“亲自推动所有事情的人”转为“建立机制、授权团队、监督方向的人”。这一步是反单点依赖的关键。

11.4 把 SOP 变成平台功能

当某个 SOP 重复执行 2-3 次后，就可以考虑沉淀成平台能力。例如：

- 多次出现任务过大，就做子任务系统。
- 多次出现新人不知道怎么参与，就做新人任务标签和教学入口。
- 多次出现成员退出，就做交接记录和任务释放机制。
- 多次出现验收不清，就做验收清单。

这样平台不是凭空设计，而是从真实运营问题中生长。

12. 总结

OpenRD 的长期价值不只在开发某一个工具，而在于建立一套可以持续把真实患者需求转化为技术协作成果的机制。

这个机制要解决四件事：

1. 找到真实需求。
2. 把需求翻译成任务。
3. 让不同能力的人都能参与。
4. 让项目在人员流动中持续推进。

因此，组织治理、子任务系统和 SOP 不是附属工作，而是平台的核心能力。

如果 OpenRD 能先用这套机制管理自身开发，并不断复盘改进，就能形成一个可复制的公益开源项目协作模板。未来平台上的每一个需求，都可以沿用这套流程完成从需求、任务、开发、验收到运维的完整闭环。